

# Linear Recurrence

## 常系数齐次线性递推

### 简介

常系数齐次线性递推数列（又称为 C-finite 或 C-recursive 数列）是常见的一类基础的递推数列。

对于数列  $a_n$  和其递推式

其中  $c_i$  不全为零，我们的目标是在给出初值  $a_0, \dots, a_{k-1}$  和递推式中的  $k$  后求出  $a_n$ 。如果  $k=1$ ，我们想要更快速的算法。

这里  $a_n$  被称为  $k$  阶的常系数齐次线性递推数列。

### Fiduccia 算法

Fiduccia 算法使用多项式取模和快速幂来计算，时间为  $O(k^3 \log n)$ ，其中  $M(k)$  表示两个次数为  $k$  的多项式相乘的时间。

**算法：**构造多项式  $P(x)$  和  $Q(x)$ ，那么

其中定义  $\langle f, g \rangle$  为内积。

**证明：**我们定义  $A$  的友矩阵为

我们定义多项式  $f(x)$  和  $g(x)$

观察到

且

我们将这个递推用矩阵表示有

可知  $A$  的第一行为  $(c_0, \dots, c_{k-1})$ ，根据矩阵乘法的定义得证。

### 表示为有理函数

对于上述数列  $a_n$  一定存在有理函数

且， $\frac{1}{x}$ 。我们称其为「有理函数」是因为它是「多项式」。

**证明：**对于  $\frac{1}{x}$  和  $\frac{1}{x^2}$  的系数定义，这几乎就是形式幂级数「除法」的定义，

我们只需要令

那么根据  $\frac{1}{x}$  的定义，必然有  $\frac{1}{x^2} = -\frac{1}{x^3}$ 。

## Bostan–Mori 算法

### 计算单项

我们的目标仍然是给出上述多项式  $\frac{1}{x^2}$ ，求算  $\frac{1}{x^2}$ 。

Bostan–Mori 算法基于 Graeffe 迭代，对于上述多项式  $\frac{1}{x^2}$  有

因为分母  $x^2$  是偶函数，所以子问题只需考虑其中的一侧

我们付出两次多项式乘法的代价使得问题至少减少为原先的一半，而当  $n=1$  时显然有  $\frac{1}{x} = -\frac{1}{x^3}$ ，时间复杂度同上。

### 计算连续若干项

目标是给出上述多项式  $\frac{1}{x^2}$ ，求算  $\frac{1}{x^2}$ 。下面的计算中我们只需考虑对答案「有影响」的系数，这是 Bostan–Mori 算法的关键。

我们不妨假设  $\frac{1}{x^2} = -\frac{1}{x^3}$ ，否则我们也可以通过一次带余除法使问题回到这种情况。

我们先考虑更简单的问题：

我们需要求出  $\frac{1}{x}$  然后作一次乘法并取出  $\frac{1}{x^2}$  的系数。令  $\frac{1}{x} = -\frac{1}{x^3}$  那么我们只需求出

就可以还原出  $\frac{1}{x}$ 。进而我们只需求出  $\frac{1}{x^2}$  再和  $\frac{1}{x}$  作一次乘法即可求出  $\frac{1}{x^3}$ 。

上面的算法虽然已经可以工作，但是每一次的递归的时间复杂度与  $n$  相关，我们希望能至少在递归求算时摆脱  $n$ ，更具体的，我们先考虑求算  $\frac{1}{x^2}$ ，考虑

我们需要求出

那么对于  $\frac{1}{x^2}$  而言，我们只需求出

这是因为

我们知道  $f(x)$  和  $f(-x)$  的奇偶性是一样的，所以

这样我们可以写出伪代码

但是只有这个算法还不够，我们需要重新找到一个有理函数并求算更多系数。

找到新的有理函数表示

我们知道  $f(x)$  本身和  $f(-x)$  的一部分连续的系数比如  $a_0, a_1, a_2$  和  $b_0, b_1, b_2$ ，我们希望求出  $a_3, a_4, a_5$ ，这等价于我们要求某个  $n$  且使得  $f(x)$  的前  $n$  项与  $f(-x)$  相同。简单来说：递推关系（有理函数的分母）是不变的，我们所做的只是更换初值（有理函数的分子）。

具体的，考虑

我们现在希望将递推前进  $n$  项，那么就是

我们先用一次  $f(x)$  计算出  $a_0, a_1, a_2$ ，然后我们扩展合并出  $a_3, a_4, a_5$ ，再重新计算一个分子使得

最后我们使用形式幂级数的除法计算出  $a_3, a_4, a_5$ ，时间为  $O(n)$ 。

## 参考文献

1. Alin Bostan, Ryuhei Mori. [A Simple and Fast Algorithm for Computing the  \$n\$ -th Term of a Linearly Recurrent Sequence](#).

---

本页面最近更新：2024/10/30 21:50:12, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [hly1204](#), [c-forrest](#), [Enter-tainer](#), [ksyx](#), [ouuan](#), [QAQAutoMaton](#), [shuzhouliu](#), [thredreams](#), [Tiphereth-A](#)

本页面的全部内容 [在 CC BY-SA 4.0 和 SATA 协议之条款下](#)提供，附加条款亦可能应用